

# MyOwnCompiler - Day 05: Strings, Characters & Lexer Diagnostics

## Compiler Engineering Journal

### Day 5 Objective

Extend the lexer to recognize string literals, character literals, and provide meaningful diagnostics.

Compiler Pipeline

Source File

|

v

Lexer

|

v

Token Stream

|

v

Parser (Day 6)

## Features Added

- ✓ String Literals
- ✓ Character Literals
- ✓ Lexer Diagnostics
- ✓ Unterminated String Detection
- ✓ New Token Types

## String Literal Example

```
print("Hello Compiler");
```

```
PRINT  
LEFT_PAREN  
STRING  
RIGHT_PAREN  
SEMICOLON  
EOF
```

## Character Literal Example

```
'A'
```

```
CHARACTER
```

## Diagnostics

Input:

@

Output:

Lexer Error  
Line 1, Column 1

Unexpected character '@'

## Unterminated Strings

Input:

"hello

Output:

Lexer Error

Unterminated string literal

## Testing Performed

PASS - Strings  
PASS - Characters  
PASS - Comments  
PASS - Keywords  
PASS - Operators  
PASS - EOF

# Interview Notes

1. Difference between string and character literals?
2. What is lexical grammar?
3. Why are lexer diagnostics important?
4. Why is lexical analysis  $O(n)$ ?
5. What is EOF?

## Revision Sheet

Day 1 - Project Setup  
Day 2 - Basic Lexer  
Day 3 - Operators & Symbols  
Day 4 - File Input & Comments  
Day 5 - Strings, Characters, Diagnostics

Core Functions:  
advance()  
peek()  
skipWhitespace()  
skipComment()  
identifier()  
number()  
stringLiteral()  
characterLiteral()  
tokenize()

## Next Step

Day 6

Recursive Descent Parser  
Tokens -> AST